Apuntes de Estudio Fundamentales: CS50P (Introducción a la Programación con Python de Harvard)

1. Introducción al Entorno de Desarrollo y Conceptos Fundamentales

1.1. El Entorno de Trabajo: CLI, VS Code y el Intérprete

Para escribir código de manera profesional es fundamental comprender la diferencia entre un **procesador de texto** (como Microsoft Word o Google Docs) y un **editor de texto** especializado (como Visual Studio Code o VS Code). Los procesadores de texto añaden formato visual subyacente (negritas, subrayados, sangrías de párrafo y metadatos) que corrompen la ejecución del código. La programación requiere exclusivamente archivos de texto plano con la extensión .py, los cuales contienen únicamente los caracteres legibles escritos por el programador.El entorno de trabajo se divide en dos áreas principales dentro de VS Code:

1. **Editor de texto:** La ventana superior donde se redacta el código fuente línea por línea.
2. **Interfaz de Línea de Comandos (CLI / Terminal):** La ventana inferior que permite interactuar directamente con el sistema operativo mediante texto.En la CLI, el símbolo del dólar ($) representa el **prompt** o indicador que señala que el sistema está listo para recibir comandos. Para ejecutar un script de Python, el programador escribe en la terminal:

```
$ python hello.py
```

El comando code hello.py se utiliza en la CLI para crear o abrir un archivo específico en el editor.

```
+-------------------------------------------------------------
------+
| VISUAL STUDIO CODE (VS Code)
|
|
|
|
+-------------------------------------------------------------
+  |
|  | Editor de Texto (hello.py)
|  |
|  | 1  print("hello, world")
|  |
|
+-------------------------------------------------------------
+  |
|  | Interfaz de Línea de Comandos (CLI / Terminal)
|  |
|  | $ python hello.py
|  |
```

```
|   | hello, world
|   |
|
+-------------------------------------------------------------------
+   |
+-------------------------------------------------------------------
------+
```

Los ordenadores operan nativamente mediante **código binario** (secuencias físicas de ceros y unos: 0 y 1). Debido a que escribir en binario es inviable para el ser humano, utilizamos lenguajes de alto nivel como Python. El **intérprete de Python** es un programa especializado que lee el código fuente en texto plano de arriba hacia abajo y de izquierda a derecha, traduciéndolo dinámicamente a código binario para que la CPU de la computadora ejecute las instrucciones.| Herramienta / Concepto | Función y Propósito para el Programador || ------ | ------ || **VS Code** | Editor de texto plano que ofrece resaltado de sintaxis, integración de terminal y herramientas de depuración (*debugging*). || **CLI / Terminal ($)** | Interfaz basada en texto para ejecutar comandos del sistema, administrar archivos y lanzar programas rápidamente. || **Archivo .py** | Documento de texto plano que almacena las instrucciones del código fuente escritas en Python. || **Intérprete de Python** | Programa que traduce el código de alto nivel a binario (0s y 1s) línea por línea durante la ejecución. |

Consejo del Profesor (CS50 Insights):*Nunca intentes ejecutar tus programas haciendo doble clic sobre el archivo .py en el explorador de tu sistema operativo. El desarrollo profesional requiere controlar la ejecución desde la CLI para inspeccionar salidas de texto, pasar argumentos dinámicos y capturar mensajes de error.*

1.2. El Primer Programa ("Hello, World!") y Anatomía de Errores

El programa canónico universal para iniciar en cualquier lenguaje es imprimir "hello, world". En Python, se escribe de la siguiente manera:

```
print("hello, world")
```

Desglose Estructural:

- **print** : Nombre de la función predeterminada integrada (*built-in*) en el lenguaje que ejecuta la acción de mostrar texto.
- **Paréntesis ()** : Encierran los argumentos o entradas que modifican el comportamiento de la función.
- **Comillas ""** : Indican que el contenido interior debe tratarse de forma literal como texto.
- **"hello, world"** : Cadena de texto (*string*) pasada como argumento a la función.Un **efecto colateral (*side effect*)** es el resultado visible o audible que genera una función en el entorno exterior. Para print, el efecto colateral es la aparición del texto en la pantalla de la terminal.

Bugs y Anatomía de Errores

Un **bug** es cualquier error o falla no deseada en un programa. Uno de los errores más comunes al iniciar es el **SyntaxError** (Error de Sintaxis). Este ocurre cuando el

programador viola las reglas gramaticales del lenguaje. Por ejemplo, al olvidar cerrar las comillas de una cadena de texto:

```
print("hello, world)
```

Al ejecutar el código anterior, el intérprete detiene la ejecución y devuelve:

```
SyntaxError: unterminated string literal
```

Un SyntaxError impide que el intérprete procese el script, indicando que hay una cadena de texto no terminada (*unterminated string literal*). A diferencia de los errores gramaticales humanos donde el contexto permite entender el mensaje, la computadora exige una precisión sintáctica exacta.*Una vez dominada la salida estática de texto en la terminal, el siguiente paso fundamental es capturar datos dinámicos proporcionados por el usuario y almacenarlos en la memoria para procesarlos.*

2. Lección 0: Funciones y Variables

2.1. Interactividad y Almacenamiento: input(), Variables y Asignación

Para hacer un programa interactivo se utiliza la función integrada input(), la cual detiene la ejecución, muestra un cursor parpadeante en la terminal y espera a que el usuario escriba una respuesta y presione *Enter* .

```
name = input("What's your name? ")
```

Todo dato retornado por la función input() es **nativamente de tipo cadena de texto (str)** , independientemente de si el usuario ingresa letras o números.

```
       [ Valor retornado: "David" ]
                     |
                     v
  name  =  input("What's your name? ")
  [  "David"  ]  <-- Almacenado en la variable 'name'
```

El Operador de Asignación (=)

En programación, el signo igual simple (=) **no significa igualdad matemática** . Se conoce como el **operador de asignación** y funciona copiando valores exclusivamente **de derecha a izquierda** . Evalúa primero todo el código situado al lado derecho y almacena el resultado en la variable declarada al lado izquierdo.

Reglas de las Variables:

- Una variable es un contenedor etiquetado en la memoria de la computadora que guarda un valor para reutilizarlo posteriormente.
- **Legibilidad:** Aunque en matemáticas se usan nombres de una sola letra (x, y, z), en programación se deben usar nombres descriptivos (como name en lugar de n o x) aprovechando el teclado completo para que el código sea autoexplicativo y fácil de leer.

2.2. Manipulación de Strings (str) y Métodos de Limpieza

Python permite formatear e integrar cadenas de texto de múltiples formas:

1. **Concatenación con + **: Une textos literalmente sin agregar espacios automáticamente.

```
print("hello, " + name)
```

1. **Múltiples argumentos en print()** : Separados por comas. Inserta automáticamente un espacio entre cada argumento.

```
print("hello,", name)
```

1. **Format Strings (f-strings)** : Sintaxis moderna y elegante. Se antepone una f antes de la primera comilla y se incluyen las variables entre llaves {}.

```
print(f"hello, {name}")
```

Métodos Integrados de Cadenas (str)

Un **método** es una función construida dentro de un tipo de dato específico. Para invocarlo se utiliza la sintaxis del punto (variable.metodo()).

- **.strip()** : Elimina espacios en blanco sobrantes al inicio y al final del texto.
- **.capitalize()** : Convierte en mayúscula únicamente el primer carácter de toda la cadena y el resto en minúsculas.
- **.title()** : Convierte en mayúscula la primera letra de cada palabra dentro del texto.
- **.split()** : Divide una cadena en una secuencia de subcadenas basadas en un separador (por defecto, un espacio).

Cadenamiento de Métodos (Method Chaining)

Es posible encadenar múltiples operaciones en una sola línea de código, procesando el texto de derecha a izquierda:

```
# Limpia espacios, aplica formato de título y asigna a la variable
name = input("What's your name? ").strip().title()

# División de cadenas en múltiples variables
first, last = name.split(" ")
```

Evaluación de Estilo: Combinar funciones en una sola línea ahorra código, pero si la línea se vuelve excesivamente larga, se reduce la legibilidad y se dificulta la detección de errores (*debugging*).

2.3. Parámetros Posicionales vs. Parámetros Nombrados (Named Parameters)

En la documentación oficial de Python, la firma formal de la función print se presenta de la siguiente forma:

```
print(*objects, sep=' ', end='\n')
```

Tipos de Parámetros:

- **Parámetros Posicionales:** La función procesa los argumentos según el orden estricto en el que se envían. El asterisco (*objects) indica que print puede recibir cero, uno o infinitos objetos posicionales.

- **Parámetros Nombrados (*Named Parameters*):** Opciones con valores predeterminados (*default values*) que se pueden sobrescribir llamándolas por su nombre:
- sep=' ': Define el separador entre argumentos posicionales (un espacio por defecto).
- end='\n': Define el carácter final que se imprime al terminar la línea (\n es un salto de línea por defecto).

Sobrescribir Comportamientos Predeterminados:

```
# Elimina el salto de línea al final de la primera impresión
print("hello, ", end="")
print(name)

# Cambia el separador por defecto
print("hello", name, sep="???")
```

Caracteres de Escape (\)

El carácter de barra invertida (\) permite incluir caracteres especiales dentro de una cadena sin romper la sintaxis:

- \": Permite imprimir comillas dobles literales sin cerrar el texto: print("hello, \"friend\"").
- \n: Representa explícitamente un salto de línea (*newline*).

2.4. Operaciones Numéricas: Integers (int) y Floats (float)

Python incluye tipos de datos nativos para realizar cálculos matemáticos:

- **int (Entero):** Números sin punto decimal ($\dots, -2, -1, 0, 1, 2, \dots$). En Python no tienen límite estricto de tamaño en memoria.
- **float (Flotante):** Números con punto decimal (números reales). Tienen precisión finita debido a los límites físicos de representación binaria de la memoria de la computadora.

Operadores Aritméticos Core:

- Suma: +
- Resta: -
- Multiplicación: *
- División: /
- Módulo (Resto de la división): %

Conversión de Tipos (Type Casting) y Anidamiento de Funciones

Dado que input() siempre retorna cadenas de texto, sumar dos cadenas mediante + resulta en concatenación ("1" + "2" produce "12"). Para operar numéricamente se realiza un *casting* usando int() o float():

```
# Anidamiento de funciones: la función interna input() se ejecuta primero;
# su retorno se pasa como argumento a la función externa int()
x = int(input("What's x? "))
y = int(input("What's y? "))
```

```
print(x + y)
```

Redondeo y Formateo de Flotantes

La función round(number, ndigits) permite redondear valores al entero o número de decimales más cercano:

```
z = round(x / y, 2)  # Redondea el resultado a 2 decimales
```

Mediante *f-strings* es posible formatear números grandes o limitar sus decimales sintácticamente:

```
# Formato de millares con comas (p. ej., 1,000)
print(f"{z:,}")

# Formato fijando exactamente 2 posiciones decimales
print(f"{z:.2f}")
```

2.5. Creación de Funciones Propias (def), Scope y Retorno

Crear funciones personalizadas permite estructurar el código, evitar repetir lógica y abstraer problemas complejos en bloques sencillos.

Sintaxis Base y Valores por Defecto:

Se utiliza la palabra clave def seguida del nombre de la función, sus parámetros entre paréntesis y dos puntos (:):

```
def hello(to="world"):
    print("hello,", to)
```

- to="world": Define un valor por defecto. Si el programador invoca hello() sin argumentos, to tomará el valor de "world".

Alcance de Variables (Variable Scope)

El **Scope** se refiere al contexto en el que una variable existe dentro del código. Una variable definida dentro de una función (alcance local) **no es accesible** desde fuera u otra función distinta.

```
def main():
    name = input("What's your name? ")
    hello(name)

def hello(to):
    # La variable 'name' NO existe aquí.
    # Usamos el parámetro local 'to' que recibió la copia de su
valor.
    print("hello,", to)
```

La Palabra Clave return

Mientras que algunas funciones se ejecutan por sus efectos colaterales (como imprimir en pantalla), otras computan datos y devuelven un resultado al programa principal mediante return:

```
def main():
    x = int(input("What's x? "))
    print("x squared is", square(x))

def square(n):
    return n * n  # Devuelve el cálculo; equivalente a n ** 2 o
pow(n, 2)
```

Estándar de Estructura con main()

En programas profesionales de Python, se organiza la lógica principal dentro de una función main() y se llama explícitamente al final del archivo. Esto soluciona el problema de orden de lectura del intérprete, permitiendo definir funciones auxiliares abajo en el archivo sin provocar errores de definición.

```
def main():
    name = input("What's your name? ")
    hello(name)

def hello(to="world"):
    print("hello,", to)

# Invocación explícita al final del archivo para iniciar la
ejecución
main()
```

Una vez estructurado el programa en funciones legibles, es momento de dotarlo de inteligencia permitiéndole tomar decisiones dinámicas mediante estructuras condicionales.

3. Lección 1: Condicionales

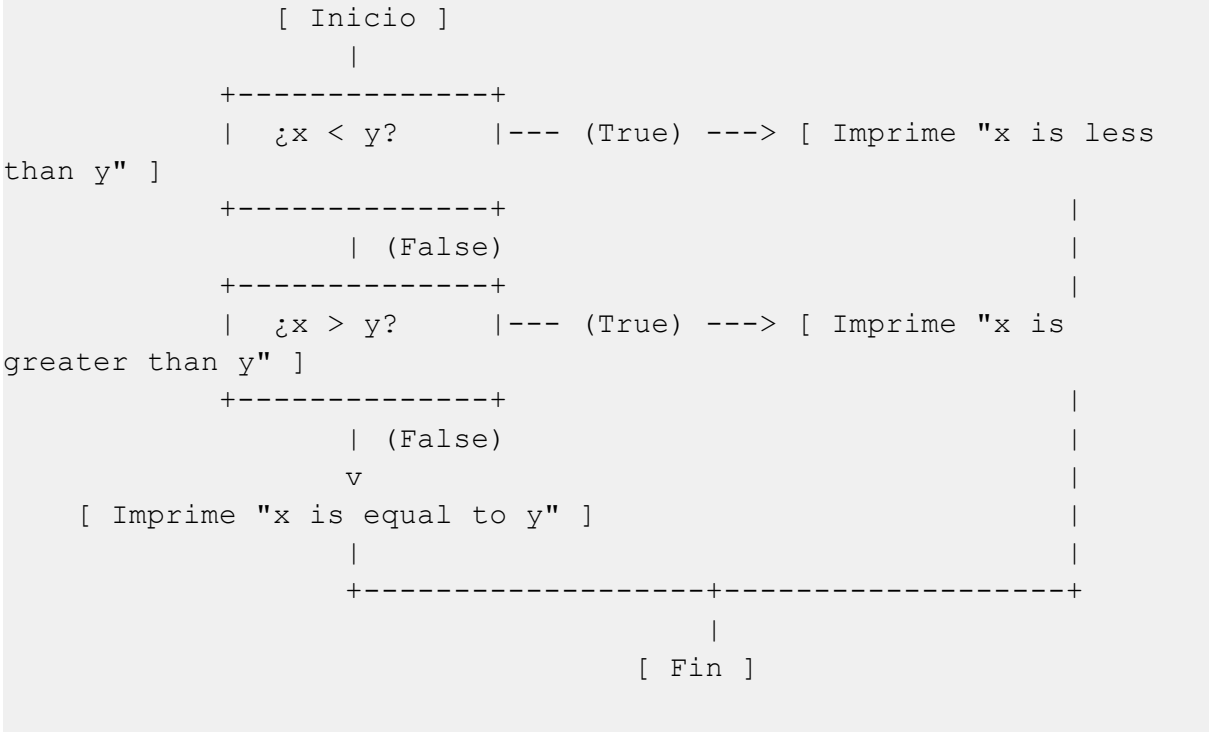
3.1. Toma de Decisiones y Operadores de Comparación

Las expresiones condicionales permiten evaluar preguntas lógicas y alternar el flujo del programa en función del resultado. Una **Expresión Booleana** es cualquier declaración lógica que se evalúa strictly como **True** (Verdadero) o **False** (Falso).| Operador | Significado | Ejemplo de Expresión Booleana || ------ | ------ | ------ || > | Mayor que | x > y || >= | Mayor o igual que | x >= y || < | Menor que | x < y || <= | Menor o igual que | x <= y || == | Igual a (Comparación estricta) | x == y || != | No igual a (Diferente de) | x != y |
Aviso de Sintaxis: Confundir el operador de asignación = con el operador de comparación == es un error común que genera fallos sintácticos o lógicos.

3.2. Estructuras de Control: if, elif y else

Las estructuras de control determinan el flujo de ejecución (*control flow*) a través de bifurcaciones lógicas.

```
if x < y:
    print("x is less than y")
elif x > y:
    print("x is greater than y")
else:
    print("x is equal to y")
```



Análisis de Eficiencia Sintáctica:

- Usar múltiples bloques if independientes obliga al sistema a evaluar **todas** las preguntas secuencialmente, independientemente de si la primera ya fue verdadera.
- La estructura if / elif / else garantiza **exclusividad mutua** . Si una condición inicial es verdadera, el programa ejecuta su bloque de código y salta inmediatamente al final de toda la estructura, ahorrando procesamiento al evitar preguntas innecesarias.
- La sentencia else actúa como un atrapalotodo (*catch-all*) para casos residuales, eliminando la necesidad de preguntas lógicas redundantes al final (p. ej., evitar evaluar elif x == y cuando las opciones < y > ya han sido descartadas).

3.3. Lógica Combinada (and, or) y Simplificación Pythonic

Las palabras clave and y or permiten unir múltiples expresiones booleanas:

- and: Requiere que **ambas** expresiones sean verdaderas.
- or: Requiere que **al menos una** expresión sea verdadera.

Comparaciones Encadenadas Estilo Pythonic

En lugar de escribir comparaciones largas unidas por and, Python permite encadenar directamente rangos matemáticos legibles:

```
# Lógica tradicional
if score >= 90 and score <= 100:
```

```
    print("Grade: A")

# Lógica Simplificada "Pythonic"
if 90 <= score <= 100:
    print("Grade: A")
```

Operaciones de Paridad con Módulo (%) y Retornos Booleanos Directos

Para determinar si un número es par o impar, evaluamos el resto de su división entre 2 usando la operación módulo (%):

```
# Evolución sintáctica para verificar si un número es par
def is_even(n):
    # Opción 1: Tradicional de 4 líneas
    # if n % 2 == 0:
    #     return True
    # else:
    #     return False

    # Opción 2: Expresión condicional concisa
    # return True if n % 2 == 0 else False

    # Opción 3: Pythonic en una sola línea (Retorna directamente
el resultado Booleano)
    return n % 2 == 0
```

3.4. Control de Patrones Multicriterio: match / case

Introducido en versiones recientes de Python, la estructura match / case simplifica la evaluación de una misma variable contra múltiples valores posibles (similar al switch en otros lenguajes):

```
name = input("What's your name? ")

match name:
    case "Harry" | "Hermione" | "Ron":
        print("Gryffindor")
    case "Draco":
        print("Slytherin")
    case _:
        print("Who?")
```

Características Clave:

- **Operador Tubería (|):** Actúa como un or lógico unificando múltiples casos dentro de un mismo bloque de ejecución.
- **Comodín Guion Bajo (_):** Representa el caso por defecto (*default wildcard*). Equivale al else de un condicional y captura cualquier valor no especificado previamente.*Una vez dominada la toma de decisiones para bifurcar el código, el siguiente fundamento es la automatización de tareas repetitivas mediante bucles.*

4. Lección 2: Bucles (Loops)

4.1. Repetición Basada en Condiciones: El Bucle while

El bucle while repite un bloque de código **mientras** una condición booleana se mantenga como verdadera (True).

```
i = 0
while i < 3:
    print("meow")
    i += 1  # Incremento de estado (Sintaxis equivalente a i = i + 1)
```

Para evitar un **bucle infinito accidental** (que ocurre si la condición nunca cambia a False y puede colgar el sistema), siempre se debe actualizar la variable de control dentro del cuerpo del bucle.**Consejo del Profesor (CS50 Insights):***Si accidentalmente ejecutas un bucle infinito en la CLI y tu terminal queda atrapada imprimiendo texto sin control, presiona la combinación de teclas* ***Control-C (Ctrl+C)*** *. Este atajo envía una señal de interrupción directa al sistema operativo para cancelar inmediatamente la ejecución del proceso.*

Patrón de Validación de Entradas (while True)

Para forzar a un usuario a ingresar datos válidos, se utiliza intencionadamente un bucle infinito controlado con las sentencias break y continue:

```
def get_number():
    while True:
        n = int(input("What's n? "))
        if n > 0:
            break  # Interrumpe y sale inmediatamente del bucle actual
        else:
            continue  # Salta el resto del bloque y reevalúa la condición
    return n
```

4.2. Repetición Basada en Secuencias: El Bucle for y range()

El bucle for se utiliza para iterar sobre una secuencia predefinida de elementos o un número exacto de iteraciones mediante la función range(n).

```
# Iteración explícita sobre una lista de números
for i in [0, 1, 2]:
    print("meow")

# Iteración limpia con range() que genera valores de 0 hasta n-1 (0, 1, 2)
for _ in range(3):
    print("meow")
```

Convención del Guion Bajo (_)

Cuando la variable de control del bucle no va a ser utilizada expresamente dentro de la lógica interior, la convención de diseño en Python impone nombrar la variable como un guion bajo simple (_). Esto le indica a otros desarrolladores que la variable solo sirve para contar las iteraciones.

Alternativa Sucinta: Multiplicación de Cadenas

Python permite replicar cadenas de texto mediante el operador de multiplicación *:

```
print("meow\n" * 3, end="")
```

Nota de diseño: Se especifica end="" explícitamente en el print porque "meow\n" * 3 ya genera tres saltos de línea al final de cada palabra. De no incluir end="", print añadiría su salto de línea predeterminado (\n), dejando una línea en blanco indeseada al final de la salida.

4.3. Estructuras de Datos Contenedoras: Listas (list)

Una **Lista** es una secuencia ordenada de valores de cualquier tipo almacenados dentro de corchetes . Las listas utilizan un sistema de indexación base cero (*zero-indexed*).

```
students = ["Hermione", "Harry", "Ron"]

# Acceso por posición de índice
print(students[0])  # Hermione
print(students[1])  # Harry
print(students[2])  # Ron
```

Formas de Iteración sobre Listas:

```
# 1. Iteración directa sobre los elementos (Forma preferida y
legible)
for student in students:
    print(student)

# 2. Iteración por posición utilizando índices numéricos con len()
y range()
for i in range(len(students)):
    print(i + 1, students[i])
```

4.4. Mapeo Clave-Valor: Diccionarios (dict) y Estructuras Anidadas

Un **Diccionario (dict)** relaciona directamente claves (*keys*) con valores (*values*), estructurándose mediante llaves {}.

```
# Mapeo simple de Nombre -> Casa
students = {
    "Hermione": "Gryffindor",
    "Harry": "Gryffindor",
    "Ron": "Gryffindor",
    "Draco": "Slytherin",
```

```
}

# Búsqueda directa por Clave
print(students["Hermione"])  # Gryffindor

# Al iterar un diccionario con 'for', por defecto se iteran sus
CLAVES
for student in students:
    print(student, students[student], sep=", ")
```

Estructuras Anidadas Complejas (Lista de Diccionarios)

Para representar información multidimensional de objetos del mundo real, se agrupan diccionarios dentro de una lista:

```
students = [
    {"name": "Hermione", "house": "Gryffindor", "patronus":
"Otter"},
    {"name": "Harry", "house": "Gryffindor", "patronus": "Stag"},
    {"name": "Ron", "house": "Gryffindor", "patronus": "Jack
Russell terrier"},
    {"name": "Draco", "house": "Slytherin", "patronus": None},
]

for student in students:
    print(student["name"], student["house"], student["patronus"],
sep=", ")
```

La Palabra Clave None

En Python, **None** es un tipo de dato especial que representa oficialmente la ausencia o inexistencia de un valor. En el ejemplo anterior, señala de forma limpia y explícita que Draco no posee un patronus conocido.

4.5. Construcción Multidimensional: Algoritmos con Bucles Anidados

Los problemas de construcción gráfica bidimensional (como imprimir rejillas o bloques al estilo Mario) requieren el uso de bucles anidados: un bucle externo encargándose de las filas y un bucle interno gestionando los caracteres/columnas.

```
def print_square(size):
    # Bucle exterior para cada fila en la cuadrícula
    for i in range(size):
        # Bucle interior para cada carácter dentro de la fila
actual
        for j in range(size):
            print("#", end="")
        # Imprime una línea en blanco al terminar la fila para
saltar a la siguiente
        print()
```

```
# Alternativa abstraída aprovechando multiplicación de cadenas:
def print_square_simplified(size):
    for i in range(size):
        print("#" * size)
```

Construir algoritmos complejos incrementa las probabilidades de enfrentarse a imprevistos durante la ejecución. Por ello, es imperativo diseñar código defensivo que controle los errores de forma elegante.

5. Lección 3: Excepciones

5.1. Taxonomía de Errores: Sintaxis vs. Tiempo de Ejecución (Runtime)

Los errores en Python se dividen en dos categorías fundamentales:

1. **Errores de Sintaxis (SyntaxError):** Infracciones del lenguaje descubiertas por el intérprete antes de arrancar la ejecución. Deben ser corregidos de forma manual por el programador directamente en el archivo fuente.
2. **Excepciones o Errores de Tiempo de Ejecución (Runtime Errors):** Fallos imprevistos que ocurren mientras el programa ya está corriendo, causados comúnmente por entradas inesperadas del usuario o datos corruptos.

Excepciones Comunes:

- **ValueError :** Ocurre cuando una función recibe un argumento con el tipo correcto pero con un valor o formato inapropiado para la conversión. Ejemplo: ejecutar int("cat").
- **NameError :** Se produce al intentar utilizar una variable que jamás ha sido asignada o que no existe en el scope actual debido a una falla previa en el flujo de ejecución.

5.2. Defensa del Código: Bloques try, except, else y finally

Python proporciona una estructura defensiva completa para capturar y procesar excepciones sin que el programa colapse catastróficamente.

- **try** : Encierra la línea o bloque reducido de código sospechoso que podría lanzar una excepción.
- **except** : Captura e identifica una excepción específica, ejecutando código de rescate si ocurre el error.
- **else** : Bloque opcional que se ejecuta **únicamente si el bloque try finalizó con éxito total** (sin lanzar ninguna excepción).
- **finally** : Bloque opcional que **se ejecuta siempre e incondicionalmente** , independientemente de si ocurrieron errores o no. Es ideal para liberar recursos, cerrar conexiones o emitir un estado final.

```
def main():
    x = get_int()
    print(f"x is {x}")

def get_int():
    while True:
        try:
```

```
            x = int(input("What's x? "))
        except ValueError:
            print("x is not an integer")
        else:
            # Se ejecuta solo si int() no lanzó ValueError
            return x
        finally:
            # Se ejecuta SIEMPRE al final de cada iteración del
bloque try/except
            print("Attempt evaluation complete.")

main()
```

Principio de Mínima Exposición

Como buena práctica de diseño, el bloque try debe ser **lo más corto posible** . Incluir múltiples líneas dentro de try puede enmascarar errores no intencionados, dificultando el rastreo de fallos reales.

5.3. Patrones de Re-solicitud e Invocación Explícita (raise)

Combinando while True, try, except y return, se crea una función reutilizable que insiste al usuario hasta obtener datos válidos:

```
def get_int(prompt):
    while True:
        try:
            # Si int() tiene éxito, 'return' retorna el valor E
interrumpe el bucle e instintivamente la función entera
            return int(input(prompt))
        except ValueError:
            # Informa al usuario y repite el ciclo dinámicamente
            print("Please enter a valid integer.")

# Uso limpio en el programa principal
x = get_int("What's x? ")
```

Invocación Explícita de Excepciones con raise

No solo el intérprete de Python puede generar excepciones; el desarrollador puede **provocar intencionadamente** una excepción propia cuando se violan las reglas de negocio mediante la palabra clave **raise** :

```
def set_age(age):
    if age < 0:
        # Lanza voluntariamente una excepción ante un valor
numérico inválido
        raise ValueError("Age cannot be a negative number")
    print(f"Age set to {age}")

try:
```

```
    set_age(-5)
except ValueError as error:
    print(f"Invalid input captured: {error}")
```

Una vez que el código individual es defensivo y seguro, la modularidad exige no reescribir soluciones comunes desde cero, aprovechando código de terceros o librerías del sistema.

6. Lección 4: Librerías y Módulos

6.1. Reusabilidad de Código: Importación de Módulos y Paquetes

Una **Librería** es un conjunto de código previamente escrito por otros desarrolladores o por el propio equipo que permite extender la funcionalidad del lenguaje y evitar "reinventar la rueda".

- **Módulo:** Un archivo individual de Python que contiene funciones y clases reutilizables.
- **Paquete:** Una colección organizada de múltiples módulos.Existen dos sintaxis primarias de importación:

```
# 1. Importa el módulo completo
import random
coin = random.choice(["heads", "tails"])

# 2. Importa exclusivamente una función específica al namespace
local
from random import choice
coin = choice(["heads", "tails"])
```

6.2. Módulos Estándar Esenciales

Python incluye la *Standard Library* (Librería Estándar), módulos preinstalados listos para usarse:

1. Módulo random

- random.choice(seq): Selecciona un elemento aleatorio de una secuencia/lista.
- random.randint(a, b): Genera un número entero aleatorio entre a y b (ambos inclusive).
- random.shuffle(seq): Desordena en sitio (*in-place*) los elementos de una lista.

2. Módulo statistics

- statistics.mean(data): Calcula el promedio o media aritmética de una lista numérica.

3. Módulo sys

Proporciona interacción directa con el intérprete y el sistema operativo.

- sys.argv: Lista que almacena los argumentos pasados al script desde la línea de comandos (donde sys.argv0 es siempre el nombre del propio archivo).
- sys.exit(msg): Detiene y finaliza la ejecución del programa de forma inmediata, opcionalmente imprimiendo un mensaje.

6.3. Paquetes de Terceros y pip

pip es el administrador de paquetes oficial de Python. Permite instalar paquetes externos alojados en índices como PyPI (*Python Package Index*).

```
$ pip install requests
```

Por ejemplo, usando la librería de terceros requests y el módulo integrado json, es posible conectar con servidores web e interrogar APIs remotas de manera limpia.*Con la integración de librerías externas y la expansión de la base de código, probar manualmente cada función resulta inviable. Se requiere de pruebas automatizadas sistemáticas.*

7. Lección 5: Pruebas Unitarias (Unit Tests)

7.1. Filosofía de Pruebas e Identificación de Regresiones

Las **Pruebas Unitarias** son programas escritos para probar código personalizado función por función. Garantizan la validez del programa en el presente y previenen las **regresiones** (fallos no intencionados introducidos en funciones antiguas al actualizar o refactorizar el código en el futuro).Para verificar condiciones de forma nativa se utiliza la palabra clave assert:

```
from calculator import square

def test_square():
    assert square(2) == 4
    assert square(3) == 9
```

Si una afirmación de assert evalúa a False, Python lanza un error de tipo AssertionError y detiene la prueba.

7.2. Automatización de Pruebas con pytest

pytest es un marco de trabajo (*framework*) de terceros que automatiza la ejecución masiva de pruebas unitarias sin requerir la escritura manual de bloques complejos de verificación.

Convenciones de Uso:

1. El archivo de prueba debe llamarse obligatoriamente test_*.py o *_test.py (por ejemplo, test_calculator.py).
2. Las funciones dentro del archivo de prueba deben iniciar con el prefijo test_.

```
# test_calculator.py
import pytest
from calculator import square

def test_positive():
    assert square(2) == 4
    assert square(3) == 9

def test_negative():
    assert square(-2) == 4
    assert square(-3) == 9
```

```
def test_zero():
    assert square(0) == 0

# Verificación de Excepciones lanzadas intencionadamente
def test_str():
    with pytest.raises(TypeError):
        square("cat")
```

Para ejecutar todas las pruebas del directorio de manera automatizada en la CLI:

```
$ pytest test_calculator.py
```

Verificada la lógica y robustez del programa, el paso siguiente es garantizar la permanencia de los datos más allá de la vida en memoria RAM de la ejecución actual.

8. Lección 6: Entrada y Salida de Archivos (File I/O)

8.1. Persistencia de Datos: Memoria Volátil vs. Almacenamiento en Disco

- **Memoria Volátil (RAM):** Es ultra rápida, pero su contenido se destruye por completo en cuanto el programa finaliza o se apaga el equipo.
- **Almacenamiento en Disco (Persistente):** Conserva la información de forma permanente organizándola en archivos y carpetas dentro del sistema de archivos.Para manipular archivos en Python se utiliza la función integrada open() especificando el modo de operación:
- "r": Lectura (*Read*). Modo por defecto.
- "w": Escritura (*Write*). Sobrescribe totalmente el archivo si ya existe o lo crea si no.
- "a": Anexar (*Append*). Agrega nuevo contenido al final del archivo existente sin borrar lo previo.

8.2. Manejo Seguro de Archivos: La Sentencia with

Olvidar cerrar un archivo abierto puede acarrear corrupción de datos o pérdidas de memoria. La sentencia with actúa como un **Gestor de Contexto (*Context Manager*)** , cerrando automáticamente el archivo una vez finalizada la tarea, incluso si ocurren excepciones.

```
name = input("What's your name? ")

# Abre o crea 'names.txt' en modo anexar
with open("names.txt", "a") as file:
    file.write(f"{name}\n")

# Lectura limpia e iteración línea por línea
with open("names.txt", "r") as file:
    for line in file:
        # rstrip() elimina el salto de línea extra que trae el
archivo
        print("hello,", line.rstrip())
```

8.3. Estructuras CSV e Imágenes

Para manejar datos estructurados organizados en filas y columnas, Python incluye el módulo csv:

- **csv.reader / csv.writer :** Trabajan manipulando listas de valores.
- **csv.DictReader / csv.DictWriter :** Mapean directamente cada columna a las claves de un diccionario, ofreciendo máxima legibilidad.

Lectura con csv.DictReader:

```
import csv

students = []

with open("students.csv") as file:
    reader = csv.DictReader(file)
    for row in reader:
        students.append({"name": row["name"], "home":
row["home"]})
```

Escritura Estructurada con csv.DictWriter:

```
import csv

name = input("What's your name? ")
home = input("Where's your home? ")

with open("students.csv", "a") as file:
    writer = csv.DictWriter(file, fieldnames=["name", "home"])
    # Opcional: Escribir encabezados si el archivo es nuevo
    # writer.writeheader()
    writer.writerow({"name": name, "home": home})
```

Adicionalmente, archivos binarios complejos (como imágenes .gif o .png) son procesados persistentemente utilizando librerías especializadas como PIL (Pillow).*El procesamiento de archivos planos estructurados conduce frecuentemente a la necesidad de validar que la información textual siga un formato o patrón estricto.*

9. Lección 7: Expresiones Regulares (Regular Expressions)

9.1. Definición y Validación de Patrones con re

Las **Expresiones Regulares (Regex)** son patrones de búsqueda definidos sintácticamente que permiten validar, extraer o reemplazar fragmentos de texto dentro de secuencias de caracteres. Se utiliza el módulo re.| Símbolo | Significado y Comportamiento || ------ | ------ || . | Cualquier carácter excepto una nueva línea || * | 0 o más repeticiones del patrón previo || + | 1 o más repeticiones del patrón previo || ? | 0 o 1 repetición del patrón previo || {m,n} | De m a n repeticiones del patrón previo || ^ | Coincidencia obligatoria al inicio de la cadena || $ | Coincidencia obligatoria al final de la cadena || | Conjunto de caracteres permitidos (p. ej., a-zA-Z0-9_) || ^ | Conjunto complementario (coincidir con

cualquier carácter EXCEPTO estos) || \d | Abreviatura para cualquier dígito decimal (0-9) || \w | Abreviatura para cualquier carácter de palabra (a-zA-Z0-9_) || \s | Abreviatura para caracteres de espacio en blanco |

9.2. Validación, Extracción y Reorganización de Datos

Las funciones clave dentro del módulo re son:

1. **re.search(pattern, string)** : Escanea el texto buscando la primera coincidencia que contenga el patrón.
2. **re.match(pattern, string)** : Evalúa coincidencias únicamente desde el inicio del texto.
3. **re.fullmatch(pattern, string)** : Exige que la totalidad de la cadena coincida exactamente con el patrón de inicio a fin.
4. **re.sub(pattern, repl, string)** : Sustituye las coincidencias del patrón por un nuevo texto de reemplazo (útil para limpieza de datos).

Grupos de Captura (), Extracción y el Operador Morsa (:=)

```
import re

email = input("What's your email? ").strip()

# El operador morsa (:=) asigna el resultado de re.search() a
'matches'
# y simultáneamente evalúa su valor de verdad en el bloque 'if'
if matches := re.search(r"^([^@]+)@([^@]+)$", email):
    print(f"Username: {matches.group(1)}")
    print(f"Domain: {matches.group(2)}")
```

Reorganización y Limpieza de Entradas con re.sub() y re.fullmatch():

```
import re

url = input("URL: ").strip()

# Limpia la URL dejando solo el nombre de usuario de Twitter/X
username = re.sub(r"^(https?://)?(www\.)?twitter\.com/", "", url)
print(f"Username: {username}")

# Validación estricta con re.fullmatch
code = input("Code: ").strip()
if re.fullmatch(r"[A-Z]{3}-\d{4}", code):
    print("Valid Product Code Format")
else:
    print("Invalid Code Format")
```

Dominados los patrones y el procesamiento procedimental de datos, es momento de escalar a una de las abstracciones más potentes del desarrollo: la modelación de entidades mediante Objetos.

10. Lección 8: Programación Orientada a Objetos (POO)

10.1. Paradigmas de Programación y Fundamentos de POO

Un **Paradigma de Programación** es un estilo o filosofía fundamental para estructurar el código:

- **Programación Procedimental:** Estructura secuencial basada en funciones y procedimientos ejecutados paso a paso de arriba a abajo.
- **Programación Orientada a Objetos (POO):** Paradigma que agrupa datos y comportamientos en entidades conceptuales que representan objetos del mundo real.
- **Programación Funcional:** Paradigma centrado en la evaluación de funciones matemáticas puras evitando estados mutables.**Consejo del Profesor (CS50 Insights):***No existe un paradigma "perfecto" o superior a los demás. En proyectos del mundo real combinarás los tres: usarás POO para modelar tus entidades principales, estilo procedimental para el flujo principal en main() y programación funcional para filtrar colecciones de datos con elegancia.*

Conceptos Core:

- **Clase (Class):** El molde, plano de construcción o tipo de dato personalizado definido por el desarrollador.
- **Objeto (Instancia):** La realización concreta en memoria construida a partir de una clase específica.

10.2. Anatomía de una Clase: Atributos y Métodos

Una clase agrupa datos (**Atributos**) y funciones propias de la entidad (**Métodos**).

```
class Student:
    # Método dunder de inicialización (Constructor)
    def __init__(self, name, house):
        if not name:
            raise ValueError("Missing name")
        if house not in ["Gryffindor", "Hufflepuff", "Ravenclaw",
"Slytherin"]:
            raise ValueError("Invalid house")
        self.name = name
        self.house = house

    # Representación en cadena de texto personalizada del objeto
    def __str__(self):
        return f"{self.name} from {self.house}"

def main():
    # Instanciación
    student = Student("Harry", "Gryffindor")
    print(student)  # Llama implícitamente a __str__
```

- **self** : Referencia obligatoria que apunta al objeto concreto que se está creando o manipulando en ese instante.

10.3. Encapsulamiento, Propiedades y Métodos de Clase

- **Encapsulamiento:** Ocultar y proteger el acceso directo a los atributos internos de una clase. Se adopta la convención de anteponer un guion bajo _atributo para declararlo protegido/privado.
- **Propiedades (@property):** Utilizan *Getters* y *Setters* para interceptar y validar el acceso o modificación de un atributo.

La Trampa de Recursión Infinita en los Setters

Es crítico que el *Setter* almacene el dato en una variable privada interna con guion bajo (self._house). Si dentro de @house.setter intentaras asignar self.house = house, la propiedad se invocaría a sí misma indefinidamente, provocando un error fatal de recursión (*RecursionError*).

```
class Student:
    def __init__(self, name, house):
        self.name = name
        self.house = house  # Invoca automáticamente al setter de
abajo

    # Getter para 'house'
    @property
    def house(self):
        return self._house

    # Setter para 'house'
    @house.setter
    def house(self, house):
        if house not in ["Gryffindor", "Hufflepuff", "Ravenclaw",
"Slytherin"]:
            raise ValueError("Invalid house")
        # Se asigna a _house con guion bajo para evitar recursión
infinita
        self._house = house

    # Método de clase (Factory method bound a la clase en sí, no a
la instancia)
    @classmethod
    def get(cls):
        name = input("Name: ")
        house = input("House: ")
        return cls(name, house)
```

10.4. Herencia y Sobrecarga de Operadores

Herencia

Permite que clases hijas reutilicen atributos y métodos de una clase padre, reduciendo duplicidad:

```
class Wizard:
```

```
    def __init__(self, name):
        if not name:
            raise ValueError("Missing name")
        self.name = name

class Student(Wizard):  # Inherits from Wizard
    def __init__(self, name, house):
        super().__init__(name)  # Invoca la inicialización de la
clase padre
        self.house = house
```

Sobrecarga de Operadores

Modifica el comportamiento de operadores matemáticos convencionales (+, -, etc.) mediante métodos dunder especiales cuando operan sobre objetos personalizados:

```
class Vault:
    def __init__(self, galleons=0, sickles=0):
        self.galleons = galleons
        self.sickles = sickles

    # Sobrecarga el operador '+'
    def __add__(self, other):
        galleons = self.galleons + other.galleons
        sickles = self.sickles + other.sickles
        return Vault(galleons, sickles)
```

Con la abstracción completa de objetos dominada, el paso final consiste en explorar herramientas sintácticas avanzadas para perfeccionar la fluidez en el desarrollo.

11. Lección 9: Conceptos Avanzados (Et Cetera)

11.1. Sintaxis Sucinta: Comprehensions y Funciones Funcionales

List Comprehensions y Dictionary Comprehensions

Permiten construir y filtrar colecciones en una sola línea elegante:

```
students = [
    {"name": "Hermione", "house": "Gryffindor"},
    {"name": "Harry", "house": "Gryffindor"},
    {"name": "Draco", "house": "Slytherin"},
]

# List Comprehension: [salida for elemento in iterable if
condicion]
lowercase_names = [s["name"].lower() for s in students if
s["house"] == "Gryffindor"]

# Dictionary Comprehension: {clave: valor for elemento in iterable
if condicion}
```

```
gryffindor_map = {s["name"]: s["house"] for s in students if
s["house"] == "Gryffindor"}
```

Ayudantes Funcionales (map, filter, lambda)

- map(func, iterable): Aplica una función a cada elemento de una colección.
- filter(func, iterable): Filtra elementos devolviendo solo los que cumplen una condición dada.
- lambda: Permite construir funciones anónimas en línea de una sola expresión:

```
# Filtra Gryffindors usando una función lambda
gryffindors = filter(lambda s: s["house"] == "Gryffindor",
students)
```

11.2. Tipado Estático y Documentación Formal

Type Hints e Inspección Estática con mypy

Python es dinámicamente tipado, pero permite agregar **Anotaciones de Tipo (*Type Hints*)** para prevenir errores de tipo antes de ejecutar el programa:

```
def meow(n: int) -> str:
    return "meow\n" * n
```

Usando herramientas de análisis estático como mypy en la terminal, se pueden detectar incoherencias de tipos antes de la ejecución:

```
$ mypy meow.py
```

Docstrings

Cadenas multilinea de documentación escritas con triple comilla """..."""" colocadas al inicio de un módulo o función para describir formalmente su propósito, parámetros y retornos:

```
def meow(n: int) -> str:
    """
    Meow n times.

    :param n: Number of times to meow
    :type n: int
    :return: A string of n meows
    :rtype: str
    """
    return "meow\n" * n
```

11.3. Línea de Comandos Avanzada y Generadores

Análisis Avanzado de CLI con argparse

El módulo argparse simplifica el procesamiento de argumentos de la línea de comandos, generando mensajes de ayuda automáticos:

```
import argparse
```

```
parser = argparse.ArgumentParser(description="Meow n times")
parser.add_argument("-n", default=1, help="number of times to
meow", type=int)
args = parser.parse_args()

for _ in range(args.n):
    print("meow")
```

Generadores y la Palabra Clave yield

Un **Generador** produce valores secuencialmente según se necesiten sobre la marcha, en lugar de cargarlos simultáneamente en la memoria RAM, siendo la técnica ideal para procesar volúmenes masivos de datos:

```
def sheep(n):
    for i in range(n):
        yield "🐑" * (i + 1)  # Retorna el valor actual y pausa la
ejecución hasta la siguiente iteración
```

12. Síntesis Final y Hoja de Ruta del Estudiante

Matriz de Aplicación Real por Lección

Lección,Concepto Núcleo CS50P,Aplicación Práctica en la Industria
0,Funciones y Variables,"Construcción de algoritmos base, limpieza e higienización de entradas de usuario."
1,Condicionales,"Control de flujo lógico, validación de reglas de negocio y bifurcación de procesos."
2,Bucles y Estructuras de Datos,Recorrido e iteración sobre grandes volúmenes de datos en diccionarios y listas anidadas.
3,Excepciones,Construcción de programas tolerantes a fallos con manejo defensivo de errores de ejecución.
4,Librerías y Módulos,Reutilización masiva de código de la comunidad y consumo de APIs remotas.
5,Pruebas Unitarias,Garantía de calidad del software mediante integración continua y prevención de regresiones.
6,Entrada/Salida de Archivos,"Persistencia de datos local, análisis y generación de reportes estructurados CSV."
7,Expresiones Regulares,"Validación estricta de formularios, filtrado de texto y raspado de datos (scraping)."
8,Programación Orientada a Objetos,"Modelado de dominios complejos del mundo real de manera modular, mantenible y escalable."
9,Conceptos Avanzados,"Optimización de memoria, análisis sintáctico formal y desarrollo de herramientas avanzadas de CLI."

Mensaje Final y Hoja de Ruta de Continuidad

El dominio de la programación con Python no se alcanza simplemente leyendo sobre sus conceptos, sino aplicándolos activamente en la resolución de problemas reales. Cada lección cubierta en este documento constituye un bloque de construcción diseñado para integrarse armónicamente con los demás.Tu siguiente objetivo fundamental debe centrarse en:

1. Resolver los **Problem Sets** semanales asociados a cada lección para afianzar la memoria muscular del código.
2. Construir proyectos propios combinando las lecciones (por ejemplo, una herramienta de CLI que consuma una API externa, procese los datos en memoria mediante POO, los valide con expresiones regulares y guarde los resultados persistentemente en un archivo CSV respaldado por pruebas unitarias automatizadas).El vocabulario técnico y la capacidad defensiva adquiridos en este curso te capacitan para enfrentarte a cualquier disciplina técnica, humanística o científica. La construcción constante de proyectos es la clave para transformarse de un aprendiz de Python en un programador competente y versátil en el mundo real.